

AI 辅助内核开发 多智能体工作流（定制版）

基于当前工作区真实工件定制： workflow 文档、 workflow.yaml 、 demo issue 、 debug 记录、 cover letter 与 checkpatch 输出。

Local Docs

Real Artifact Paths

Demo Patch-Ready

本次引用文件

ai-assisted-kernel-
development-multi-agent-
workflow.md

workflow.kernel-multi-
agent.example.yaml

artifacts/demo-riscv-kvm-mm-
kconfig/...

定制输入材料与落盘位置

方案文档

ai-assisted-kernel-development-multi-agent-workflow.md
riscv-arm-x86-gap-multi-agent-workflow.md
openclaw-linux-riscv-kernel-contribution-plan.md

控制面配置

workflow.kernel-multi-agent.example.yaml
artifacts/.../state/workflow.yaml

问题工件

issue-brief.md
plans/design.md
review/spec-round-2.md

Patch 工件

series-cover-letter.md
checkpatch.txt
get-maintainer.txt

workspace root

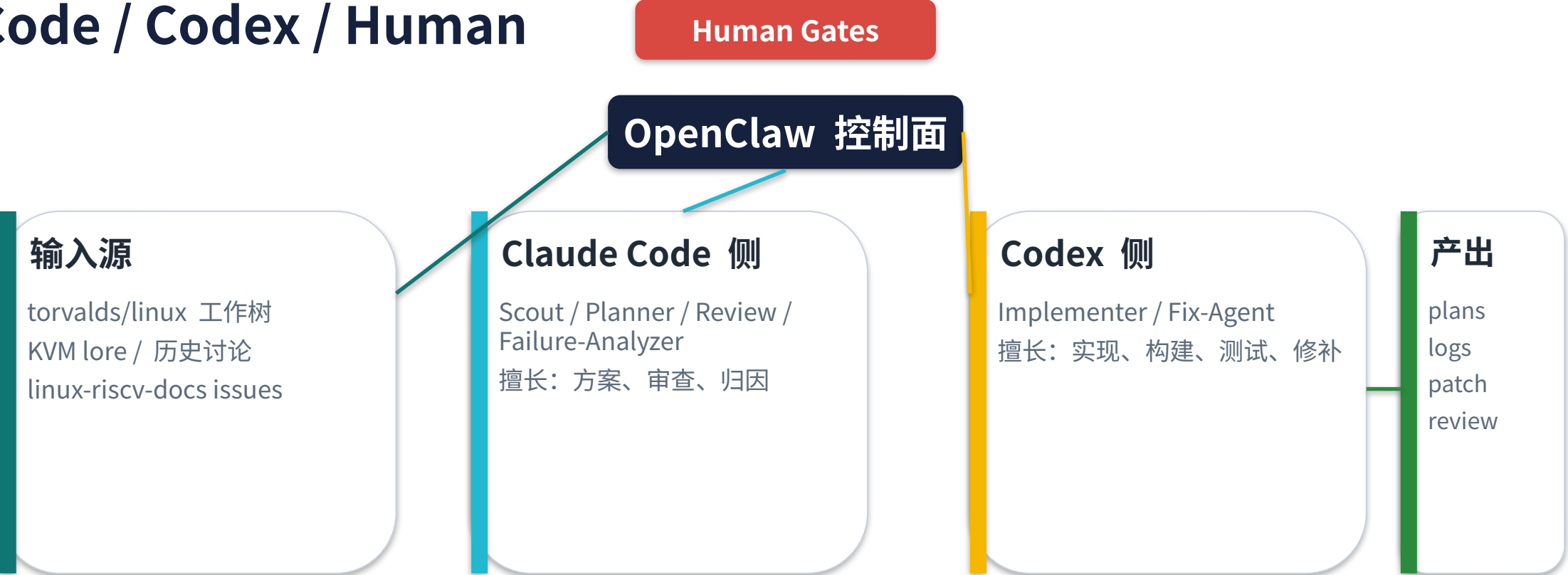
artifacts/demo-riscv-kvm-mmukconfig/
discover/ plans/ review/
debug/ patch/ state/ logs/

为什么这版更“定制”

不再只讲抽象架构，而是直接映射到当前仓库中的真实路径、真实 issue_id 和真实工件名称

汇报时可直接点击到对应文件继续深挖

本地架构图： OpenClaw / Claude Code / Codex / Human



这张图直接对应本仓库文档中的“控制面 + 执行 Agent + 人工 Gate”设计，而不是泛化 AI workflow 模板。

现有 workflow.yaml 片段：状态机真正规约在哪里

workflow.yaml excerpt

```
version: 1
name: linux-kernel-multi-agent-mvp
description: >-
  面向 Linux 内核贡献的多智能体工作流示例。聚焦 MVP：探索 -> 规划 ->
  实现 -> 审核 -> 调试 -> 再审核 -> patch-ready。
metadata:
  date: 2026-04-14
  owner: human-maintainer
  references:
    - ai-assisted-kernel-development-multi-agent-workflow.md
    - kernel-multi-agent-role-prompts-claude-codex.md
workspace:
  root: ./artifacts/${ISSUE_ID}
  issue_id_source: env:ISSUE_ID
  directories:
    discover: discover
    plans: plans
```

读法

workspace 决定工件目录；controller 决定重试与 stop；
human_gates 决定何时必须人工审批

这意味着什么

Agent 可以被替换，但工件契约与状态机不应漂移
系统稳定性来自文件规范，不来自单次 prompt 灵感

汇报价值

领导可看 Gate 与风险；工程师可看 outputs 与
stop_conditions；执行者可看 agent inputs/outputs

真实示例： Issue Brief + Design Plan

issue-brief.md

Issue Brief

Issue ID: demo-riscv-kvm-mmU-kconfig
Title: riscv/kvm: make MMU dependency explicit in Kconfig
dependency/help text
Type: low-risk Kconfig consistency fix
Subsystem: RISC-V / KVM
Priority: medium
Confidence target: high

design.md (file-level change list)

1. 只修改 `arch/riscv/kvm/Kconfig`
2. 让 `CONFIG\_RISC\_V\_KVM` 的 dependency 与 help text 都显式体现 host KVM 依赖 MMU
3. 不修改运行时代码
4. 不修改默认配置，除非验证表明现有默认配置文本会产生明显歧义

File-level change list
- Modify: `arch/riscv/kvm/Kconfig`

这页想传达的关键信息

这不是“随便找个 bug”，而是一个被明确定义为低风险、单文件、可回滚的 demo issue

Design plan 明确写出 must-not-do boundary：不引入新符号、不碰 runtime、不做无关 cleanup

这类 issue 最适合做 MVP 首轮演示

真实示例：为什么 Round 1 被打回

failure-analysis-round-1.md

Failure phenomenon

- 构建 / 配置解析未失败
- 失败发生在规格审查：实现没有完整满足 design.md 中 “dependency + help text 一并澄清” 的目标

Most likely root cause

- Implementer 把问题理解成 “dependency 修正” 而不是 “配置语义整体澄清”
- 这是规格覆盖不完整，不是运行时 bug

Alternate root causes

- design.md 对 help text 的要求不够突出
- 实现者优先保守，担心修改帮助文本会超范围

Recommended minimal fix

- 继续只修改 `arch/riscv/kvm/Kconfig`
- 在 help text 中补一句明确说明 host KVM 依赖 MMU、non-MMU 配置不适用
- 不扩大到其他 Kconfig 项

根因不是编译错误

构建 / 配置检查没炸，但规格目标要求 “dependency + help text 一并澄清”

Round 1 只完成了一半

因此需要 Debug

Debug 在这里承担 “规格缺口归因” 而不是 “修一个 crash”

汇报亮点

非常适合向听众解释：为什么生成和审核一定要允许多轮迭代

Patch-ready 不是一句话，而是一组真实工件

series-cover-letter.md

Subject: [PATCH 0/1] riscv/kvm: clarify MMU requirement in Kconfig

This demo series contains a single low-risk Kconfig consistency fix for RISC-V host KVM.

The change does not alter runtime behavior. It only makes the MMU requirement explicit in `CONFIG\_RISCV\_KVM` dependency/help text so the configuration surface matches the expected host KVM assumptions more clearly, especially when exploring non-MMU configurations.

checkpatch 结果

artifacts/.../patch/checkpatch.txt

当前 demo: total 0 errors, 0 warnings, 0 checks

get_maintainer 输出

RISC-V maintainers

KVM maintainers

linux-riscv@lists.infradead.org

kvm@vger.kernel.org

最后仍需 Gate-3

这版定制汇报会明确说明：patch-ready ≠ 自动发送

怎么把这版继续用在你的实际汇报里

适合技术评审会

展示 workflow.yaml 片段

展示 issue/design/debug/patch 实际路径

强调多轮审查与人工 Gate

适合项目推进会

把 demo issue 换成你的下一条真实 issue

把 simulated build/test 替换成真实日志

补进真实 get_maintainer / lore thread

下一步建议

增加真实 patch diff 截图或代码片段

加入成本统计页（轮次 / token / 时长）

把架构图导出成正式 SVG 后替换

这版已经绑定到当前工作区的真实文件结构